

NOT A LINK TO THE GAME. THE GAME.

A whole game, held **inside** a chain.

Every byte of the engine and its data is deployed as contract code on an EVM chain. No server holds it. Your browser reads it out of chain state, checks it against a commitment the chain itself can verify, and runs it.

[Read it off the chain](#)[How it works](#)

\$DEPTH · CONTRACT 0xc23317490b24d6e53c0137ed5d0d2ae565674bcb

[COPY](#)[explorer ↗](#)

Pull it off the chain

Nothing here is cached and nothing is served from this page. Every byte below arrives from an RPC node and is checked before it is run.

DEPTH

First-person maze

SIEGE

Top-down arena shooter

DRIFT

Vector asteroid shooter

BATTLESHIP

1v1 naval duel · the chain referees

[LIVE](#)

● IDLE

[Fullscreen](#)[Load from chain](#)

waiting.

Collect the lamps, then find the exit. Click the view to capture the mouse; WASD to move, A/D or arrows to turn, Q/E to strafe.

CHUNKS

—

BYTES ON CHAIN

—

CONTRACTS READ

—

MERKLE ROOT

—

The chain checks its own copy

This is the part an equivalent on Solana cannot do. There, a program cannot read transaction data – the bytes sit in the ledger and an RPC hands them back, so the verification lives in the page and you are trusting a webpage again.

Here the bytes are **contract code**. The ROM contract reads its own chunks with **EXTCODECOPY**, hashes them, and walks the merkle tree to its committed

root. Ask it about any chunk and the answer comes from the EVM, not from JavaScript.

● NOT RUN

Spot-check 6 chunks

verify(index, proof) – evaluated on chain.

What the loader refuses

- × **An unsealed ROM.** Before sealing, the owner can still append chunks and set any root, so matching the root would prove only that the publisher was self-consistent when you looked.
- × **A wrong, swapped, missing or duplicated chunk.** The tree is rebuilt from what actually arrived and compared with the commitment.
- × **A payload that inflates cleanly but is the wrong game.** Both the compressed and the decompressed forms are committed to, because a valid gzip stream is not evidence of anything.
- × **A malformed bundle.** The root proves the bytes were published; it says nothing about whether they are well formed.

How a game fits in a chain

01 Bundle

The engine and its data files go into one container, then gzip once over the whole thing.

02 Chunk

Cut into 24,575-byte pieces – EIP-170 caps deployed code at 24,576 and one byte is the STOP prefix that keeps the data from being executable.

03 Commit

A keccak256 merkle tree over the chunk hashes. keccak rather than SHA-256 because the EVM computes it natively, so a contract can check a proof for a few thousand gas.

04 Deploy

Each chunk becomes its own contract. The ROM records the pointers, then seals – one way, so the root becomes a commitment rather than a label.

05 Read

The browser fetches each pointer's code, rebuilds the tree, compares the root, inflates with the platform's own gzip, and compiles the WebAssembly – in that order. Never compile first.

What it costs

PAYLOAD	SIZE	AS CALldata	AS CONTRACT CODE	CONTRACTS
ZZT (1991)	200.0 KB	\$0.58	\$7	9
Wolfenstein 3D shareware	1.40 MB	\$4	\$51	60
Doom shareware (their build)	2.24 MB	\$7	\$82	96
FreeDM (free Doom deathmatch)	10.81 MB	\$32	\$395	462
Freedoom, both phases	23.03 MB	\$68	\$842	983

Calldata is the direct equivalent of writing bytes into transactions: permanent and retrievable, but invisible to contracts. Contract code costs more and is readable by the chain. Mainnet figures include the EIP-7623 data floor, which raises a data-heavy transaction to 40 gas per byte rather than 16 – costing this workload at 16 understates mainnet by two and a half times. Blobs are excluded on purpose: clients drop them after about eighteen days, so they are not storage.

Honest status

WORKING

- + Packer – bundle, gzip, chunk, keccak merkle

NOT YET

- A second ROM, to prove the reader is not special-cased to one

- + ROM contract – code storage and on-chain proof verification
- + Publisher – plan, deploy, verify each chunk, seal
- + Raw EIP-1559 signing, no wallet library
- + Loader and boot path, browser and Node
- + DEPTH – an original raycaster, engine and all, on chain
- + 207 tests, including a full publish-and-run against a real EVM

- Sound – the engine is silent by choice, not by limitation
- More levels than the one

What ships here is **DEPTH**, written from scratch for this: the raycasting renderer, the wall textures, the level and the creatures are all defined in code, and there are no asset files at all – the textures are arithmetic and the map is a string literal. That was the point. Bytes on a chain are permanent and cannot be taken down, so a payload has to be one nobody can later object to, and the surest way to get one is to write it.

☰ chainrom · read from chain state, verified by the chain · \$DEPTH 0xc233...674bcb

✕ @ChainRom_